Práctica Multiproceso: Cálculo de Primos en Java

Informe breve con capturas de ejecución reales y comparativa de tiempos

Alumno: Kevin Muñoz Córcoles	**Curso:** 2º DAM (Desarrollo Multiplataforma)
Módulo: Programación de Servicios y Procesos (PSP)	**Unidad:** UD1 — Multiproceso (ProcessBuilder)
GitHub: https://github.com/Kaik180/Curso-Dam2	**Repositorio:** Curso-Dam2

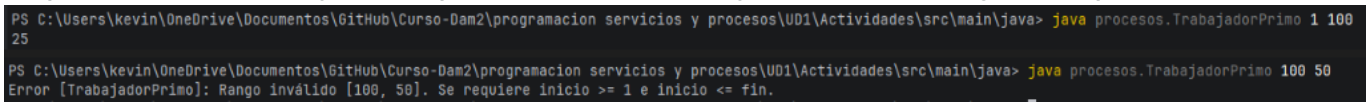
1. ¿Qué hace el programa y qué clases hemos creado?

La práctica consiste en contar cuántos números primos hay del 1 al 200.000 comparando dos formas de trabajar: en un programa normal (secuencial) o repartiendo el cálculo entre varios procesos hijos a la vez (multiproceso). Hemos creado tres clases:

- **TrabajadorPrimo.java:** Es el proceso hijo. Recibe por consola inicio y fin. Si los datos están mal, avisa por stderr y sale con código 1. Si están bien, cuenta los primos de su trozo, lo saca por stdout y sale con código 0.
- **MaestroProcesos.java:** Es el programa principal. Divide el rango en trozos iguales, lanza los hijos con ProcessBuilder, lee sus resultados con BufferedReader, espera con waitFor() y suma todos los primos.
- **PrimosSecuencial.java:** Hace la misma cuenta de forma tradicional (sin procesos) para comprobar que el número de primos coincide (17.984 primos) y comparar tiempos.

2. Capturas Reales de Ejecución en la Consola

Captura 2.1: Pruebas de TrabajadorPrimo (nominal de 1 a 100 dando 25, y control de error en rango invertido):



Captura 2.2: Ejecución de PrimosSecuencial (cuenta los 17.984 primos en 18 ms):



Captura 2.3: Ejecución de MaestroProcesos (4 subprocesos concurrentes calculando hasta 200.000):



3. Tabla Comparativa de Tiempos

Tiempos reales medidos en el equipo para el rango pedido en la práctica (hasta 200.000, encontrando 17.984 primos) y una prueba adicional con un cálculo muy pesado (hasta 10.000.000, con 664.579 primos):

Prueba / Rango	Modo de Ejecución	Procesos	Tiempo (ms)	Tiempo (s)	Conclusión rápida
Rango pedido (Hasta 200.000)	Secuencial (Normal)	1	**18 ms**	0,018 s	**El más rápido (Gana)**
	Multiproceso	2	564 ms	0,564 s	Más lento por abrir 2 Java
	Multiproceso (Práctica)	4	**298 ms**	0,298 s	Más lento por abrir 4 Java
	Multiproceso	8	1.285 ms	1,285 s	El más lento (abre 8 Java)
Prueba pesada (Hasta 10.000.000)	Secuencial (Normal)	1	3.340 ms	3,340 s	Se hace pesado (1 solo núcleo)
	Multiproceso (Óptimo)	4	**2.069 ms**	2,069 s	**Gana (ahorra 1,3 segundos)**

4. Conclusiones: Ventajas, Inconvenientes y Sobrecarga

A) ¿Por qué el multiproceso es más lento con 200.000 números? (Sobrecarga de la JVM):

Cada vez que lanzamos un proceso con ProcessBuilder ejecutando java, el sistema operativo tiene que abrir la máquina virtual de Java (JVM) completa desde cero para ese proceso. Eso supone reservar memoria RAM, cargar librerías internas de Java y preparar el entorno, lo que tarda entre **150 y 200 ms por proceso**. Como calcular los primos hasta 200.000 solo le cuesta al ordenador 18 ms, abrir 4 procesos hace que tardemos casi 300 ms en total, yendo más lento que el secuencial.

B) ¿Cuándo merece la pena usar multiproceso?:

Merece la pena cuando el cálculo es muy pesado. Al probar con **10 millones de números**, el secuencial tarda 3,3 segundos. En este caso, como el trabajo tarda bastante tiempo, el coste de arrancar las máquinas virtuales compensa totalmente: al repartir los números entre 4 procesos que corren a la vez en los núcleos del procesador, el tiempo baja a **2,06 segundos** (un ahorro de más de 1,2 segundos). Con 8 procesos no mejora más porque el procesador se satura cambiando entre tantos programas.

C) Ventajas e inconvenientes del modelo de procesos:

- **Ventaja principal (Aislamiento de memoria):** Cada proceso trabaja en su propia memoria cerrada e independiente. Si un proceso trabajador falla, da error o se cuelga, *el programa maestro sigue funcionando y no se cae*. Simplemente detecta el código de salida en waitFor().
- **Inconveniente 1 (Consumo de RAM):** Cada proceso gasta mucha memoria porque necesita una copia completa de Java funcionando.
- **Inconveniente 2 (Comunicación lenta):** Para pasarse datos entre procesos hay que leer líneas de texto por consola (stdout con BufferedReader), lo cual es más lento que usar hilos (*threads*) que comparten variables en la misma memoria.